



https://github.com/krishnm10/MarketingAdvantage_AI_v1/tree/rustPython

Act as a Senior Principal Python & AI Engineer. Conduct a brutal, zero-compromise code review of the following files. Your goal is to move this from "working code" to "world-class production-ready software."

Scope: Recursively analyze all provided files and directories. Ignore alembic/ migrations and exact file duplicates.

Review Pillars (Be Strict):

Architectural Integrity: Identify violations of SOLID principles, improper coupling, or "Leaky Abstractions" between layers (e.g., DB logic in the API layer).

Performance & Concurrency: Look for N+1 query problems, blocking I/O in async functions, CPU-bound tasks missing offloading to thread/process pools, and inefficient memory usage in AI/Data pipelines.

Production Hardening: Identify missing input validation (Pydantic/Type hinting), lack of robust error handling (no "bare excepts"), and improper logging (lack of structured logs or trace IDs).

AI/ML Standards: Check for hardcoded model paths, lack of GPU memory management, missing batching logic, and absent data validation/schema enforcement for model inputs.

Security & Secrets: Flag any hardcoded credentials, insecure defaults, or potential injection vectors.

Maintainability: Critique naming conventions, docstring quality (Google/NumPy style), and excessive cyclomatic complexity.

Output Format: Provide a Severity-Ranked List (Critical, High, Medium). For every issue, provide the Line Number, the Root Cause, and the Production-Grade Refactored Code.

I've created a detailed, severity-ranked code review as a Markdown report with concrete refactors and code snippets for each issue (Critical, High, Medium). It covers the ingestion v2 stack, RAG API, pipeline factory, and retrieval services, with line-level pointers and production-grade fixes.

You can open the attached report titled:

“World-Class Code Review – MarketingAdvantage_AI_v1 (rustPython)”

It contains:

- Critical issues: async blocking, insecure media upload path, Chroma-locked retrieval bypassing the pluggable VectorDB layer, and blocking compensation paths.
- High-severity issues: weak request validation/typing on ingestion APIs, non-batched query embedding, logging/observability gaps, and VectorDB timeout concerns.

- Medium-severity issues: ingestion service god-module, scattered env-driven config, minor dead code, AI/ML GPU-awareness gaps, and missing app-level rate limiting.

If you'd like, the next step can be to prioritize a concrete migration plan (e.g., "fix Criticals only" vs. "Critical + High") and I can generate targeted patches or PR-ready diffs for specific files.